

Endpoint Detection & Response

Deploying, Forwarding, and Validating Sysmon-Based Detection with Wazuh

Sysmon Telemetry · Detection-Gap Diagnosis · Rule Engineering · File Integrity Monitoring

Prepared by

Khizar Ul Islam (Malik)

SOC Analyst L1 Trainee

mrkhizar97@gmail.com
linkedin.com/in/khizarulislam79
github.com/Khizar-malik97

July 2026

01 Executive Summary

This lab documents building an active endpoint detection and response capability on top of a Wazuh SIEM deployment monitoring a Windows 11 endpoint. Starting from a healthy Sysmon sensor, the exercise covers verifying telemetry flow, configuring real-time File Integrity Monitoring (FIM) against a simulated persistence attempt, and configuring an automated Active Response that detects and blocks a live SMB brute-force attack launched from a Kali Linux attack host — with the full detect-to-block chain captured end-to-end.

The environment consists of a Wazuh manager (Ubuntu Server) monitoring a Windows 11 endpoint running Sysmon, attacked from a dedicated Kali Linux VM on the same lab network. Every configuration change, test action, and resulting detection shown in this report was captured directly from the live lab as the work was performed.

02 Lab Environment

Component	Detail
SIEM / Manager	Wazuh Manager v4.14 — Ubuntu Server (192.168.44.136)
Monitored Endpoint	Windows 11 Pro — agent "Khizar-Malik-Windows11"
Attack Host	Kali Linux (VMware) — same lab network as the Wazuh manager
Endpoint Telemetry	Sysmon sensor, Wazuh File Integrity Monitoring (syscheck)
Response Mechanism	Wazuh Active Response — win_route-null (automated IP block)
Capabilities Validated	Sysmon health · FIM (real-time persistence detection) · Active Response (automated attacker block)

03 Environment & Sysmon Verification

The lab began from the Wazuh Overview dashboard, which gives an at-a-glance view of agent health and alert volume across severity bands before any lab-specific configuration work started.

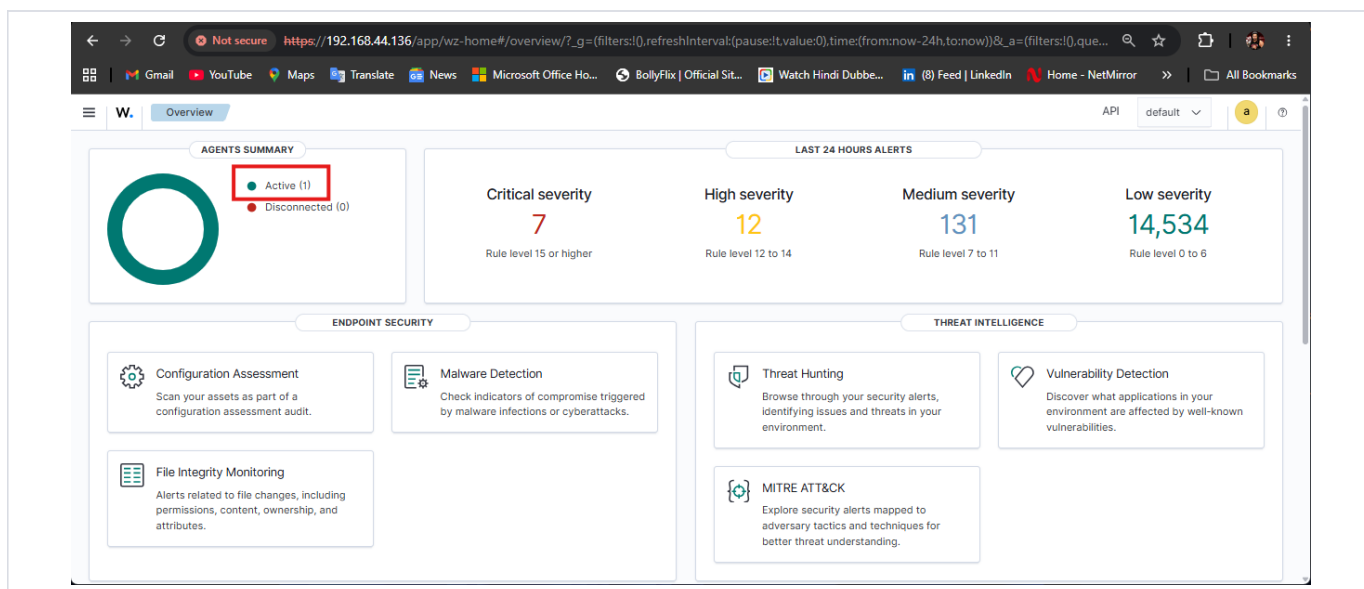


Figure 1. Wazuh Overview dashboard at the start of the lab — one active agent, with alerts already distributed across Critical, High, Medium, and Low severity bands from ongoing background monitoring.

Before trusting any downstream detection, the Sysmon sensor itself was confirmed alive and running as a Windows service on the endpoint.

COMMAND

```
Get-Service Sysmon64
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Malik> Get-Service Sysmon64

Status  Name      DisplayName
-----
Running Sysmon64 Sysmon64

PS C:\Users\Malik> |
```

Figure 2. Get-Service confirming the Sysmon64 service is installed and in a Running state on the Windows 11 endpoint.

The Wazuh agent's local configuration file was then located directly on the endpoint to confirm how it was set up to forward Sysmon telemetry to the manager.

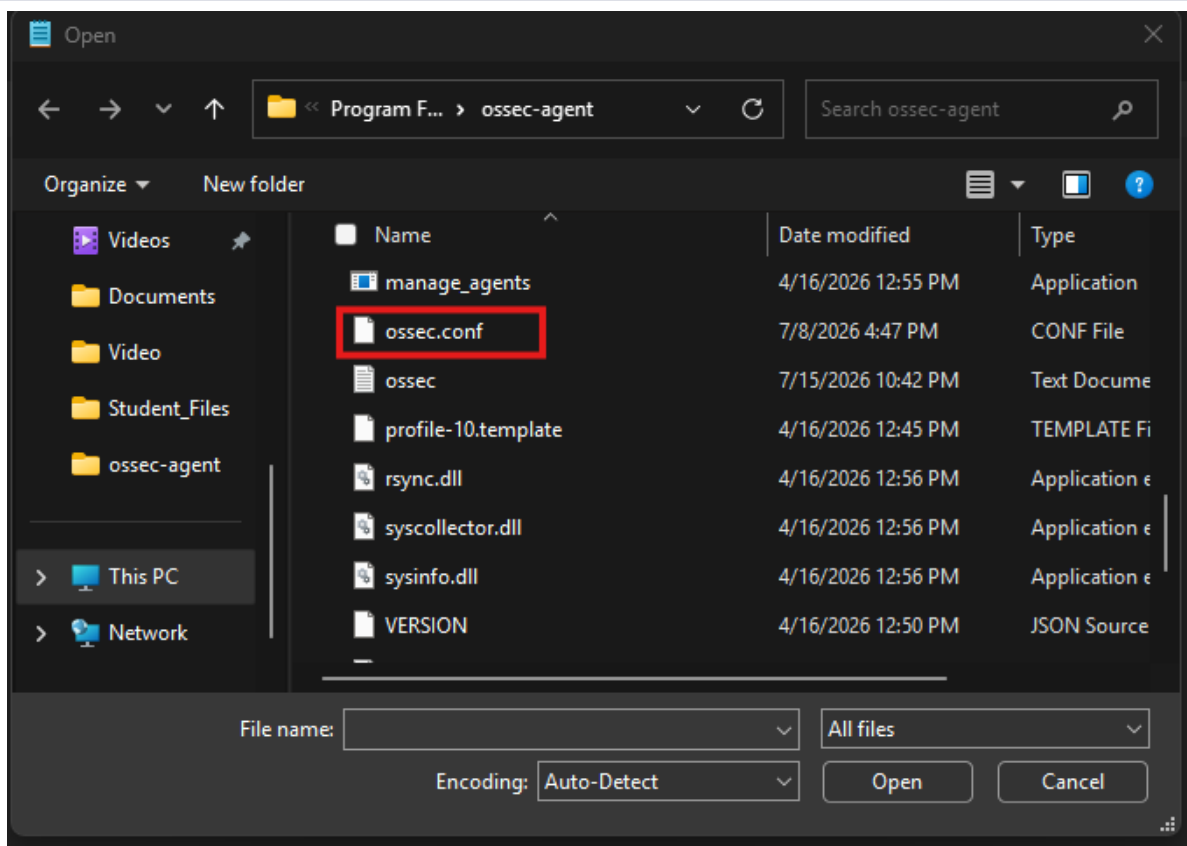


Figure 3. Locating ossec.conf inside the Wazuh agent's installation directory on the Windows endpoint.

Opening the file confirmed a `<localfile>` block already present, instructing the agent to read the Sysmon Operational event channel and forward it to the manager over an encrypted connection.

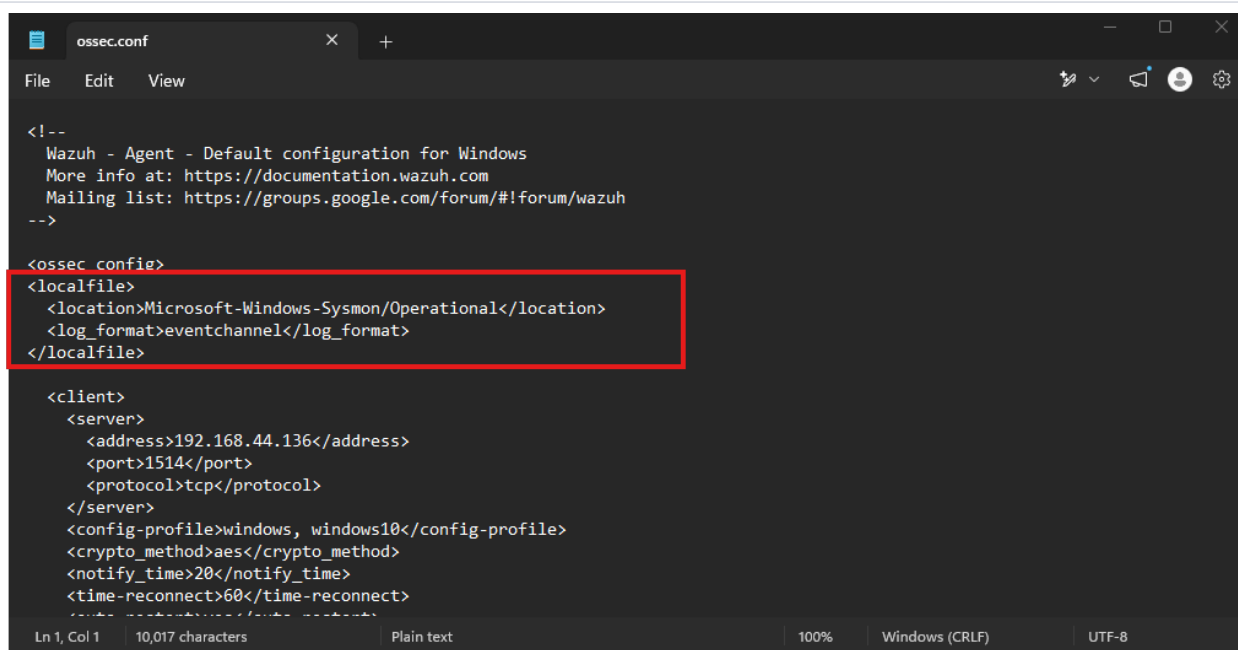


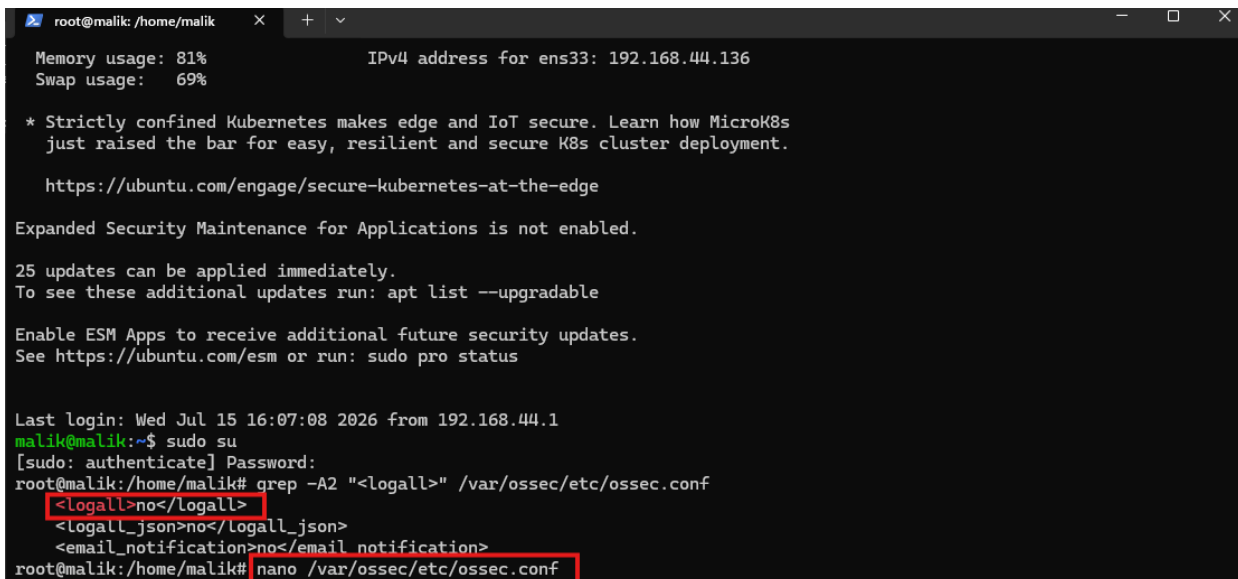
Figure 4. The Wazuh agent's ossec.conf showing the Sysmon `<localfile>` entry (Microsoft-Windows-Sysmon/Operational, eventchannel format).

04 Enabling Full Diagnostic Logging

To verify telemetry independently of whether a specific rule had matched yet, the manager's raw-event archiving setting was checked. Both `logall` and `logall_json` were disabled by default, meaning only events that already matched a rule were being retained — insufficient for confirming raw forwarding health.

COMMAND

```
grep -A2 "<logall>" /var/ossec/etc/ossec.conf
nano /var/ossec/etc/ossec.conf
```



```
root@malik: /home/malik x + ~
Memory usage: 81% IPv4 address for ens33: 192.168.44.136
Swap usage: 69%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

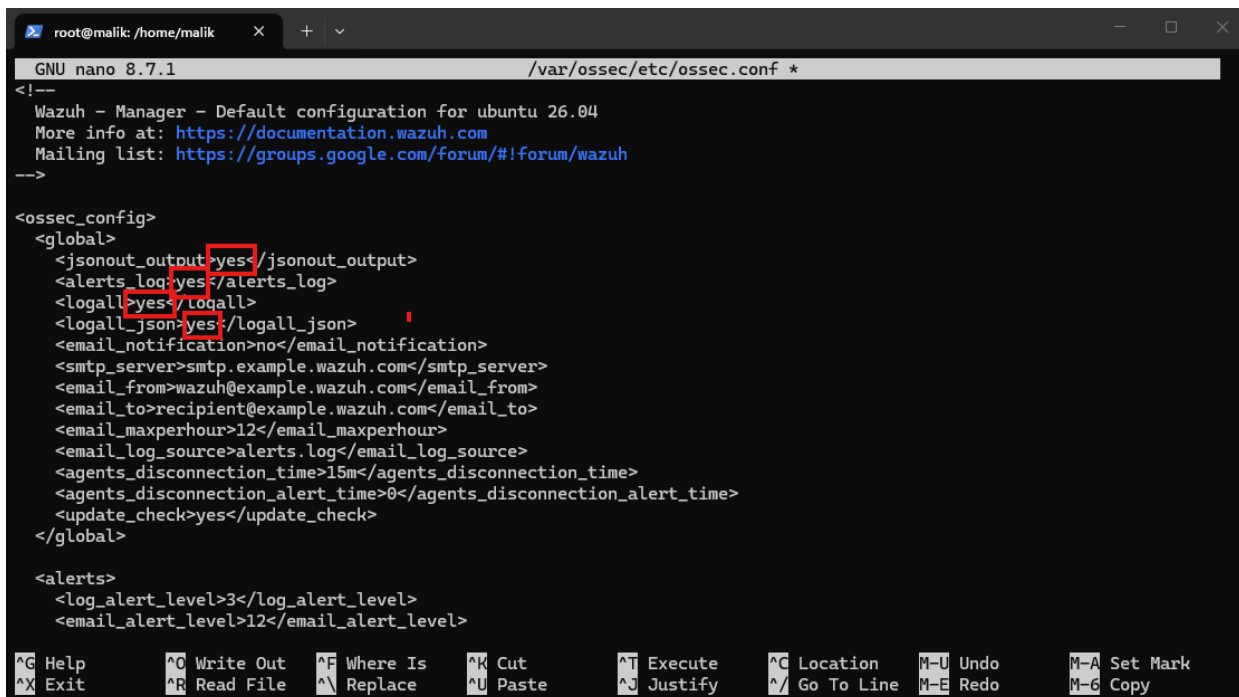
25 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Jul 15 16:07:08 2026 from 192.168.44.1
malik@malik:~$ sudo su
[sudo: authenticate] Password:
root@malik:/home/malik# grep -A2 "<logall>" /var/ossec/etc/ossec.conf
<logall>no</logall>
<logall_json>no</logall_json>
<email_notification>no</email_notification>
root@malik:/home/malik# nano /var/ossec/etc/ossec.conf
```

Figure 5. Manager terminal confirming `logall` and `logall_json` were both set to "no" by default, before opening the configuration file to change them.

Inside the editor, both settings were switched to "yes" under the `<global>` block so that every event received by the manager — not only rule matches — would be retained for verification.



```
root@malik: /home/malik
GNU nano 8.7.1 /var/ossec/etc/ossec.conf *
<!--
Wazuh - Manager - Default configuration for ubuntu 26.04
More info at: https://documentation.wazuh.com
Mailing list: https://groups.google.com/forum/#!forum/wazuh
-->

<ossec_config>
  <global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
    <logall>yes</logall>
    <logall_json>yes</logall_json>
    <email_notification>no</email_notification>
    <smtp_server>smtp.example.wazuh.com</smtp_server>
    <email_from>wazuh@example.wazuh.com</email_from>
    <email_to>recipient@example.wazuh.com</email_to>
    <email_maxperhour>12</email_maxperhour>
    <email_log_source>alerts.log</email_log_source>
    <agents_disconnection_time>15m</agents_disconnection_time>
    <agents_disconnection_alert_time>0</agents_disconnection_alert_time>
    <update_check>yes</update_check>
  </global>

  <alerts>
    <log_alert_level>3</log_alert_level>
    <email_alert_level>12</email_alert_level>
  </alerts>
</ossec_config>

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo     M-G Copy
```

Figure 6. Editing the manager's ossec.conf global block, switching jsonout_output, alerts_log, and logall to "yes".

With the change saved, the manager was restarted and a live tail was started on the raw archive log, filtered for Sysmon-related entries, to watch telemetry arrive in real time.

COMMAND

```
systemctl restart wazuh-manager
tail -f /var/ossec/logs/archives/archives.json | grep -i sysmon
```

```
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

25 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Jul 15 16:07:08 2026 from 192.168.44.1
malik@malik:~$ sudo su
[sudo: authenticate] Password:
root@malik:/home/malik# grep -A2 "<logall>" /var/ossec/etc/ossec.conf
<logall>no</logall>
<logall_json>no</logall_json>
<email_notification>no</email_notification>
root@malik:/home/malik# nano /var/ossec/etc/ossec.conf
root@malik:/home/malik# grep -A2 "<logall>" /var/ossec/etc/ossec.conf
<logall>yes</logall>
<logall_json>yes</logall_json>
<email_notification>no</email_notification>
root@malik:/home/malik# systemctl restart wazuh-manager
root@malik:/home/malik# tail -f /var/ossec/logs/archives/archives.json | grep -i sysmon
```

Figure 7. Manager restarted and a live filtered tail started on the raw archive log, ready to confirm Sysmon telemetry arriving in real time.

4.1 Generating test activity

To have fresh, attributable activity to look for in the archive log, Notepad was opened on the Windows endpoint — a simple, deliberate action that generates a Sysmon Process Create event.

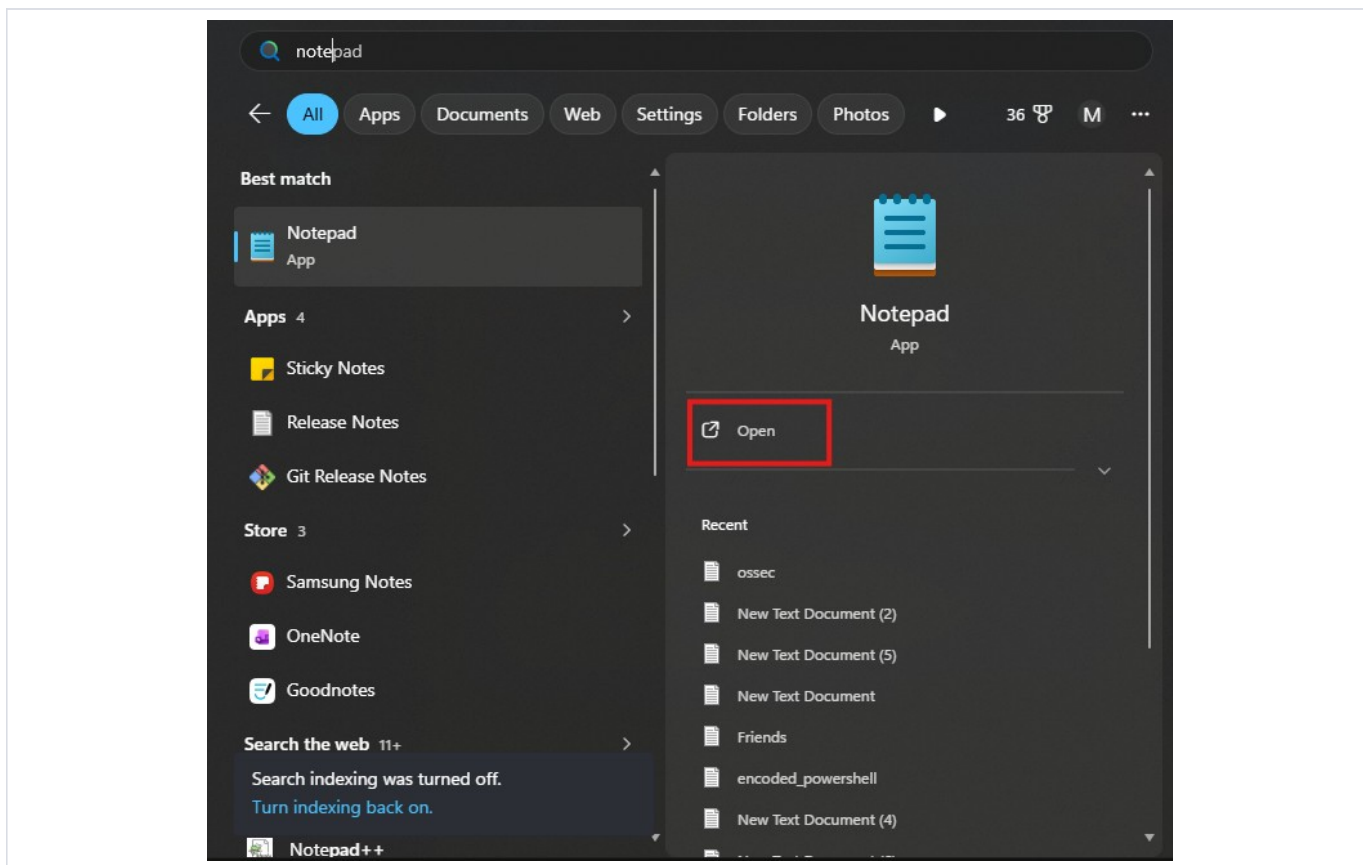


Figure 8. Opening Notepad on the Windows endpoint to generate a deliberate, attributable Sysmon Process Create event.

4.2 Reviewing captured events in the dashboard

Switching to the Wazuh dashboard's Threat Hunting view and reviewing recent events for the endpoint confirmed a healthy stream of activity being captured and scored, including a high-severity detection for an executable dropped into a location commonly used by malware — early confirmation the detection pipeline was live before the more targeted FIM and Active Response work began.

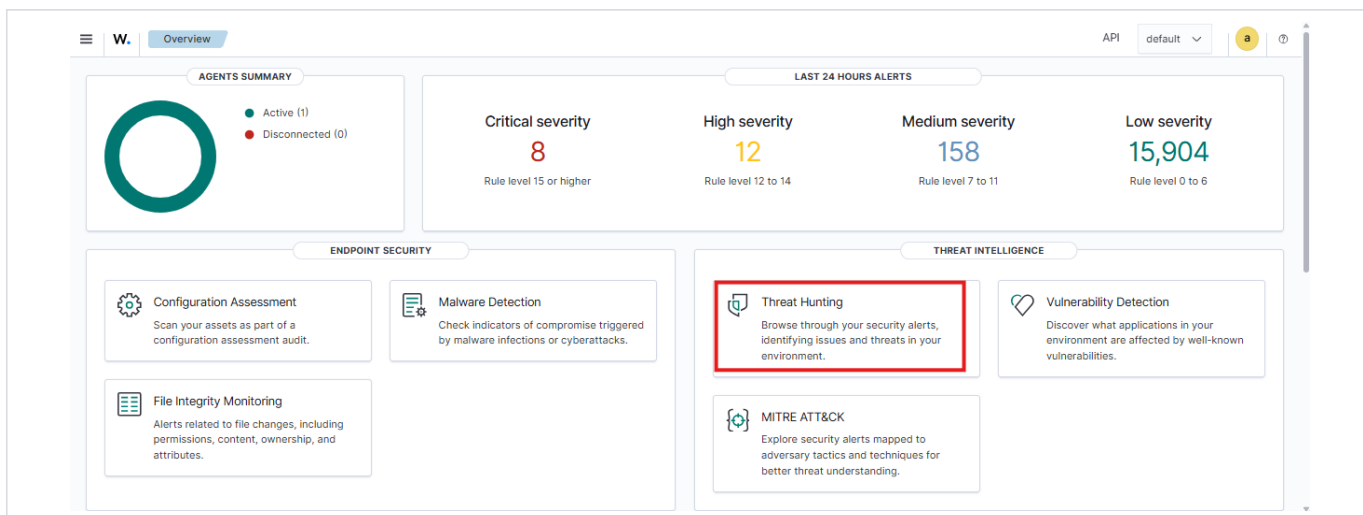


Figure 9. Threat Hunting dashboard showing a healthy stream of scored events for the endpoint, including a high-severity "executable dropped" detection.

05 File Integrity Monitoring (FIM) Configuration

With telemetry flow confirmed, attention moved to configuring real-time File Integrity Monitoring (FIM / syscheck) — Wazuh's hash- and timestamp-based tamper detection, independent of any specific process sensor. Before editing configuration, the current alert volume for the endpoint was reviewed as a baseline.

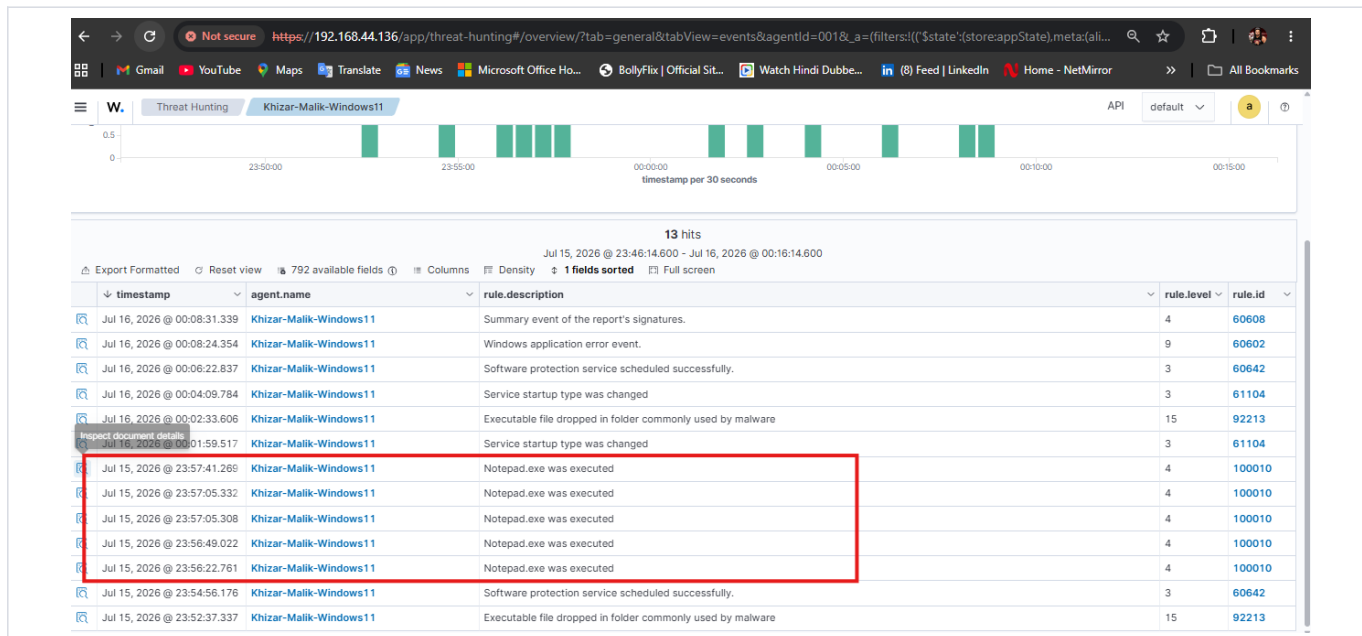


Figure 10. Baseline Threat Hunting review of recent events for the endpoint before FIM configuration changes were applied.

The agent's connection status and configuration group were confirmed directly from the manager before making any shared-configuration changes.

COMMAND

```
/var/ossec/bin/agent_control -i 001
```

```

Jul 16 16:24:59 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:24:59Z","tags":["info","s
Jul 16 16:25:01 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:25:01Z","tags":["warning"]
Jul 16 16:25:01 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:25:01Z","tags":["info","p
Jul 16 16:25:02 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:25:02Z","tags":["info","p
Jul 16 16:25:02 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:25:02Z","tags":["info","p
Jul 16 16:25:02 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:25:02Z","tags":["info","p
Jul 16 16:25:04 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:25:04Z","tags":["info","p
Jul 16 16:25:05 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:25:05Z","tags":["listenin
Jul 16 16:25:07 malik opensearch-dashboards[58399]: {"type":"log","@timestamp":"2026-07-16T16:25:07Z","tags":["info","h

root@malik:/home/malik# /var/ossec/bin/agent_control -i 001

Wazuh agent_control. Agent information:
  Agent ID:      001
  Agent Name:    Khizar-Malik-Windows11
  IP address:    any
  Status:        Active

  Operating system:  Microsoft Windows 11 Pro
  Client version:    Wazuh v4.14.5
  Configuration hash: ab73af41699f13fdd81903b5f23d8d00
  Shared file hash:   e15cd0c9f7dec03be7b82012b99d73bf
  Last keep alive:    1784220744

  Syscheck last started at: Thu Jul 16 14:07:56 2026
  Syscheck last ended at:   Thu Jul 16 14:23:40 2026

root@malik:/home/malik#

```

Figure 11. Manager confirming agent 001 (Khizar-Malik-Windows11) is Active, with prior syscheck scan timestamps visible.

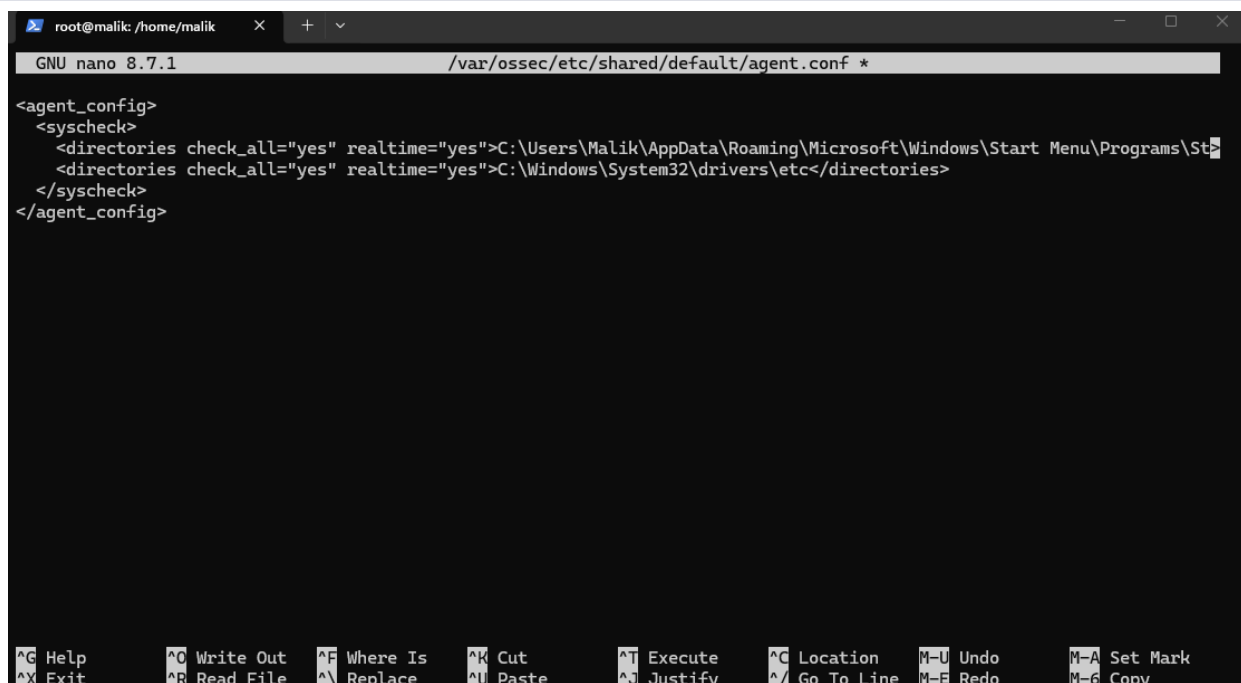
FIM was then configured to watch two locations chosen for their relevance to real attacker behaviour: the Windows Startup folder — a classic persistence location — and the hosts file, a classic target for DNS-hijacking-style tampering. Real-time monitoring was enabled on both paths.

COMMAND

```

<agent_config>
  <syscheck>
    <directories check_all="yes" realtime="yes">
      C:\Users\Malik\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\
      Startup
    </directories>
    <directories check_all="yes" realtime="yes">C:\Windows\System32\drivers\
    etc</directories>
  </syscheck>
</agent_config>

```



```
root@malik: /home/malik
GNU nano 8.7.1 /var/ossec/etc/shared/default/agent.conf *

<agent_config>
  <syscheck>
    <directories check_all="yes" realtime="yes">C:\Users\Malik\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\St
    <directories check_all="yes" realtime="yes">C:\Windows\System32\drivers\etc</directories>
  </syscheck>
</agent_config>

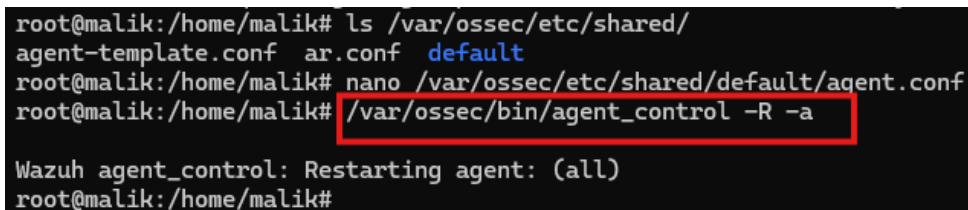
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^V Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-G Copy
```

Figure 12. FIM (syscheck) configuration added to the agent's shared configuration file, targeting the Startup folder and the hosts file with real-time monitoring enabled.

The configuration was pushed out and the agent instructed to restart and pick up the new monitoring scope.

COMMAND

```
/var/ossec/bin/agent_control -R -a
```



```
root@malik:/home/malik# ls /var/ossec/etc/shared/
agent-template.conf  ar.conf  default
root@malik:/home/malik# nano /var/ossec/etc/shared/default/agent.conf
root@malik:/home/malik# /var/ossec/bin/agent_control -R -a

Wazuh agent_control: Restarting agent: (all)
root@malik:/home/malik#
```

Figure 13. The updated configuration pushed to the agent, with Wazuh confirming the restart command was issued.

The dashboard was then set up to watch specifically for FIM-related activity on the endpoint ahead of the persistence simulation.

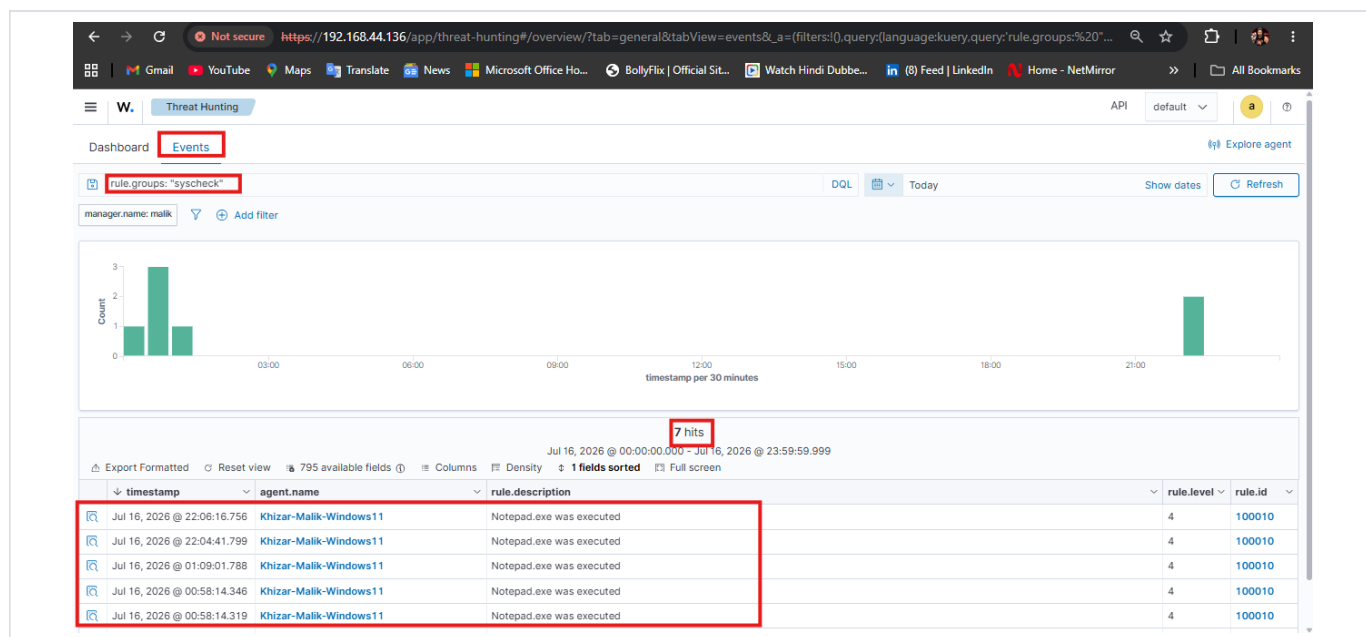


Figure 14. Threat Hunting dashboard prepared with a filtered view of the endpoint, ready to capture the upcoming File Integrity Monitoring test.

5.1 Simulating a persistence attempt

To generate a realistic, deliberate test, a copy of `calc.exe` was renamed and placed into the Startup folder — imitating a common attacker technique of disguising a malicious binary as a legitimate update file (MITRE ATT&CK T1547.001, combined with T1036 masquerading).

COMMAND

```
Copy-Item "C:\Windows\System32\calc.exe" `
"$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\`
totally_legit_update.exe"
```

```

Administrator: Windows PowerShell
to 'Enabled'] [any] [https://workbench.cisecurity.org/benchmarks/288]
F:%WINDIR%\System32\drivers\npf.sys;
F:%WINDIR%\Sysnative\drivers\npf.sys;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:avp.ch|avp.ru|nai.com;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:avp.ch|avp.ru|nai.com;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:awaps.net|mcafee.com;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:awaps.net|mcafee.com;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:microsoft.com|f-secure.com;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:microsoft.com|f-secure.com;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:sophos.com|symantec.com;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:sophos.com|symantec.com;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:my-etrust.com|viruslist.ru;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:my-etrust.com|viruslist.ru;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:networkassociates.com;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:networkassociates.com;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:kaspersky|grisoft.com;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:kaspersky|grisoft.com;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:symantecliveupdate.com;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:symantecliveupdate.com;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:clamav.net|bitdefender.com;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:clamav.net|bitdefender.com;
F:%WINDIR%\System32\Drivers\etc\HOSTS -> r:antivirus.com|sans.org;
F:%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:antivirus.com|sans.org;

PS C:\WINDOWS\system32> Remove-Item "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\totally_legit_update.exe" -Force
PS C:\WINDOWS\system32> Copy-Item "C:\Windows\System32\calc.exe" "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\totally_legit_update.exe"
PS C:\WINDOWS\system32>

```

Figure 15. PowerShell command simulating a persistence attempt — a renamed executable copied into the Startup folder to imitate a disguised malicious update.

5.2 FIM detection confirmed

Querying the dashboard for File Integrity Monitoring activity (rule group "syscheck") on the endpoint returned exactly the evidence expected: a "File added to the system" alert (Rule 554) and a matching "File deleted" alert (Rule 553) from the delete-and-recreate cycle used to produce an unambiguous, real-time result, alongside other endpoint activity captured in the same window.

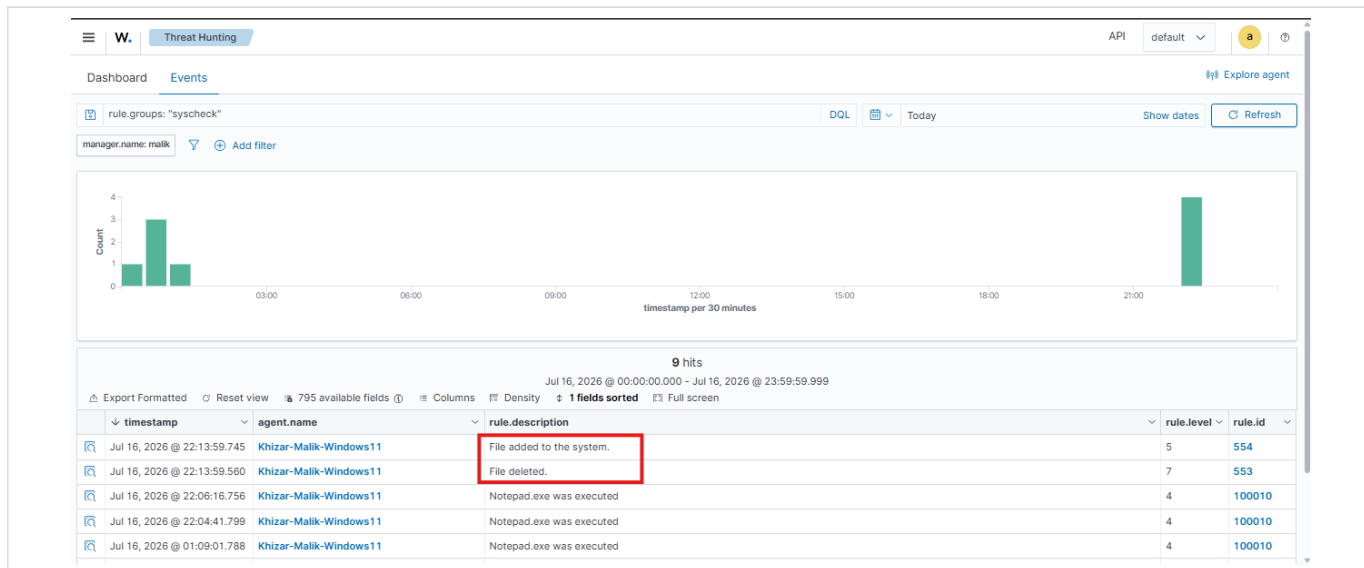


Figure 16. File Integrity Monitoring evidence — "File added to the system" (Rule 554) and "File deleted" (Rule 553) firing in the correct sequence, confirming real-time detection of the simulated persistence attempt.

Result

File Integrity Monitoring successfully detected an unauthorized file change in a monitored persistence location in

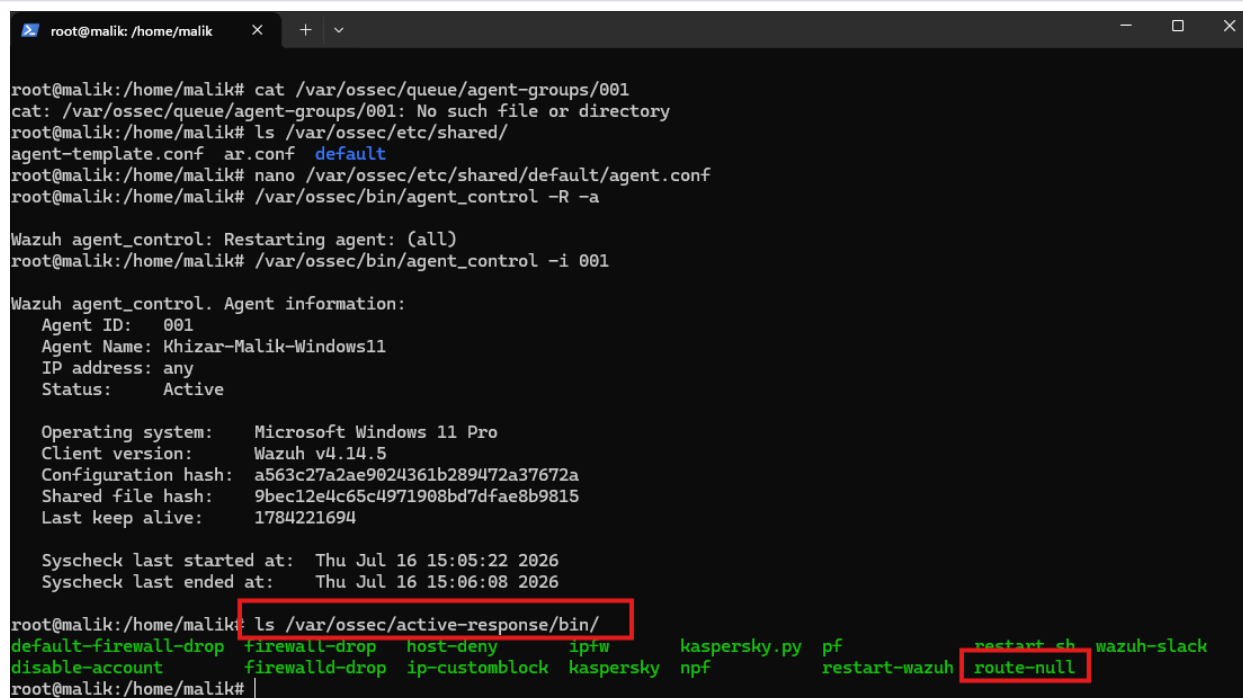
real time, independent of the process that created it — demonstrating the core tamper-detection value of EDR-style FIM.

06 Configuring Automated Active Response

With detection confirmed on both the process (Sysmon) and file-integrity layers, the lab moved to the defining capability of EDR: automated response. Wazuh ships several built-in active-response scripts; before wiring one in, the manager's and agent's available scripts were both confirmed to make sure the chosen mechanism actually existed on the Windows endpoint.

COMMAND

```
/var/ossec/bin/agent_control -i 001
ls /var/ossec/active-response/bin/
```



```
root@malik:/home/malik# cat /var/ossec/queue/agent-groups/001
cat: /var/ossec/queue/agent-groups/001: No such file or directory
root@malik:/home/malik# ls /var/ossec/etc/shared/
agent-template.conf  ar.conf  default
root@malik:/home/malik# nano /var/ossec/etc/shared/default/agent.conf
root@malik:/home/malik# /var/ossec/bin/agent_control -R -a
Wazuh agent_control: Restarting agent: (all)
root@malik:/home/malik# /var/ossec/bin/agent_control -i 001
Wazuh agent_control. Agent information:
  Agent ID: 001
  Agent Name: Khizar-Malik-Windows11
  IP address: any
  Status: Active

  Operating system: Microsoft Windows 11 Pro
  Client version: Wazuh v4.14.5
  Configuration hash: a563c27a2ae9024361b289472a37672a
  Shared file hash: 9bec12e4c65c4971908bd7dfae8b9815
  Last keep alive: 1784221694

  Syscheck last started at: Thu Jul 16 15:05:22 2026
  Syscheck last ended at: Thu Jul 16 15:06:08 2026
root@malik:/home/malik# ls /var/ossec/active-response/bin/
default-firewall-drop  firewall-drop  host-deny  iptw  kaspersky.py  pf  restart_sh  wazuh-slack
disable-account  firewallld-drop  ip-customblock  kaspersky  npf  restart-wazuh  route-null
root@malik:/home/malik#
```

Figure 17. Confirming agent connectivity and listing the manager's available active-response scripts, including route-null — the mechanism selected for automated blocking.

The equivalent check was run directly on the Windows endpoint to confirm the matching Windows executable was present, since not every active-response script has a Windows-side counterpart.

COMMAND

```
Get-ChildItem "C:\Program Files (x86)\ossec-agent\active-response\bin\"
```

```

Administrator: Windows PowerShell

f:\%WINDIR%\System32\Drivers\etc\HOSTS -> r:networkassociates.com;
f:\%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:networkassociates.com;
f:\%WINDIR%\System32\Drivers\etc\HOSTS -> r:kaspersky|grisoft.com;
f:\%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:kaspersky|grisoft.com;
f:\%WINDIR%\System32\Drivers\etc\HOSTS -> r:symantecliveupdate.com;
f:\%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:symantecliveupdate.com;
f:\%WINDIR%\System32\Drivers\etc\HOSTS -> r:clamav.net|bitdefender.com;
f:\%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:clamav.net|bitdefender.com;
f:\%WINDIR%\System32\Drivers\etc\HOSTS -> r:antivirus.com|sans.org;
f:\%WINDIR%\Sysnative\Drivers\etc\HOSTS -> r:antivirus.com|sans.org;

PS C:\WINDOWS\system32> Remove-Item "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\totally_legit_update.exe" -Force
>> Copy-Item "C:\Windows\System32\calc.exe" "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\totally_legit_update.exe"
PS C:\WINDOWS\system32> Get-ChildItem "C:\Program Files (x86)\ossec-agent\active-response\bin\"

Directory: C:\Program Files (x86)\ossec-agent\active-response\bin

Mode                LastWriteTime         Length Name
----                -
-a----            4/16/2026 12:55 PM          206000 netsh.exe
-a----            4/16/2026 12:56 PM          109856 restart-wazuh.exe
-a----            4/16/2026 12:56 PM          202416 route-null.exe

PS C:\WINDOWS\system32>

```

Figure 18. Windows-side confirmation that route-null.exe exists in the agent's active-response directory, alongside netsh.exe and restart-wazuh.exe.

6.1 Defining the response

The manager's configuration file included a commented-out placeholder for active-response definitions. This was opened to add a genuine, working configuration.

COMMAND

```
nano /var/ossec/etc/ossec.conf
```

```

root@malik:/home/malik# nano /var/ossec/etc/ossec.conf
root@malik:/home/malik#

```

Figure 19. Opening the manager's ossec.conf to add a real active-response definition in place of the default commented-out placeholder.

The existing placeholder block — inert boilerplate, not a functioning configuration — was located directly beneath the manager's predefined command definitions (including netsh).

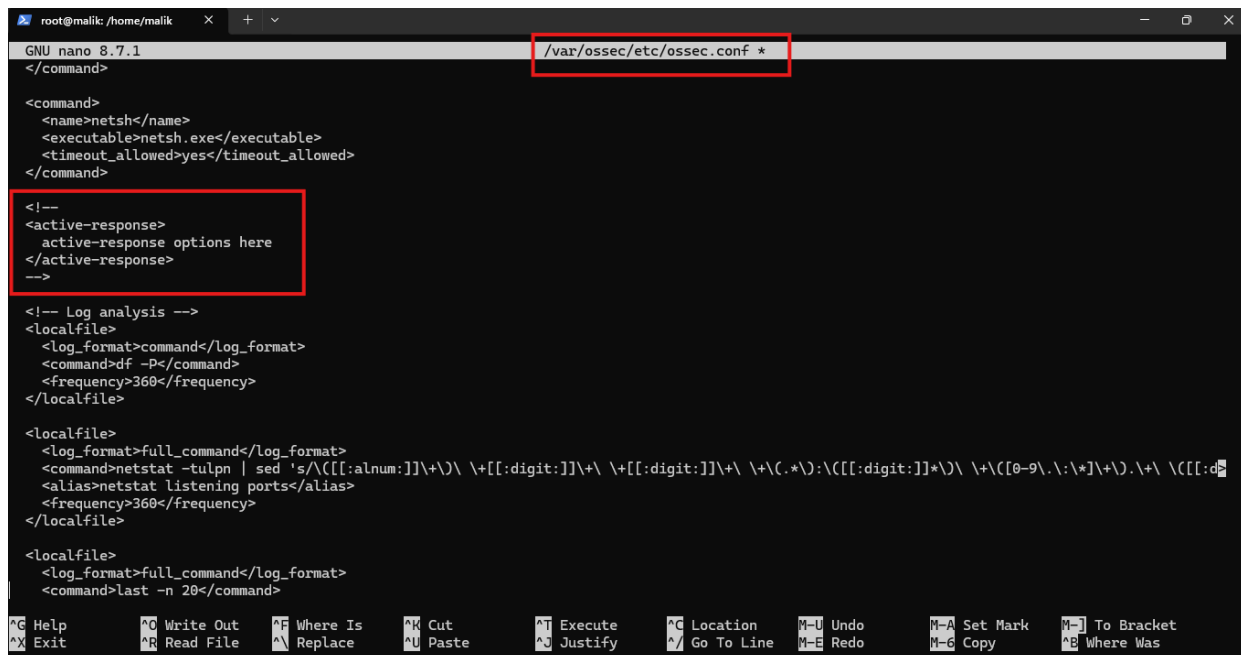


Figure 20. The default, commented-out `<active-response>` placeholder found in `ossec.conf` — present but inactive until replaced with a real definition.

A genuine active-response block was added: binding Wazuh's built-in win_route-null command to fire whenever the endpoint's Windows-specific logon-failure rules (60122 and 60115) matched, automatically null-routing the offending source for a 10-minute window.

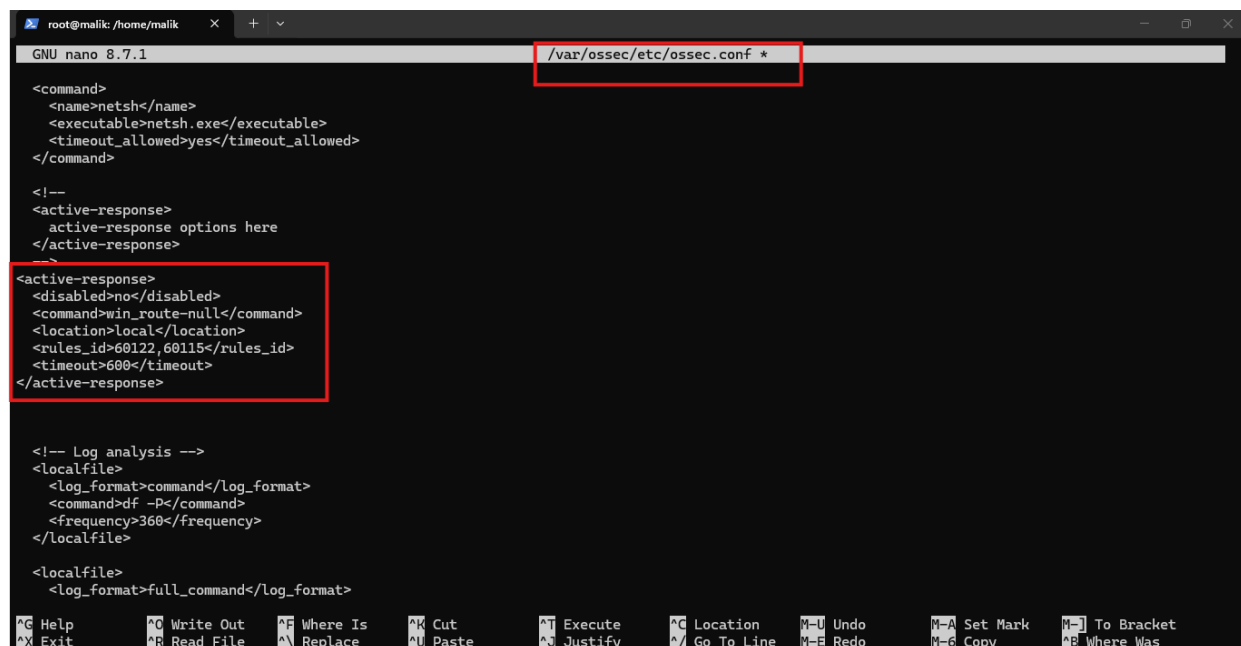


Figure 21. The real, uncommented <active-response> block added — binding win_route-null to rules 60122 and 60115 with a 600-second timeout.

The file was saved and reopened briefly to confirm the change had been written correctly before restarting the manager to load it.

COMMAND

```
nano /var/ossec/etc/ossec.conf
```

```
root@malik:/home/malik# nano /var/ossec/etc/ossec.conf
root@malik:/home/malik# systemctl restart wazuh-manager
```

Figure 22. Reopening the configuration file to verify the saved active-response block.

```
root@malik:/home/malik# nano /var/ossec/etc/ossec.conf
root@malik:/home/malik# systemctl restart wazuh-manager
root@malik:/home/malik# tail -f /var/ossec/logs/active-responses.log
```

Figure 23. Final confirmation pass on the saved ossec.conf before restarting the manager to load the new Active Response rule.

07 Attack Simulation — SMB Brute Force from Kali

To trigger the new Active Response under realistic conditions, a live brute-force attack was launched from a dedicated Kali Linux VM against the Windows endpoint's SMB service, on the same lab network as the Wazuh manager.

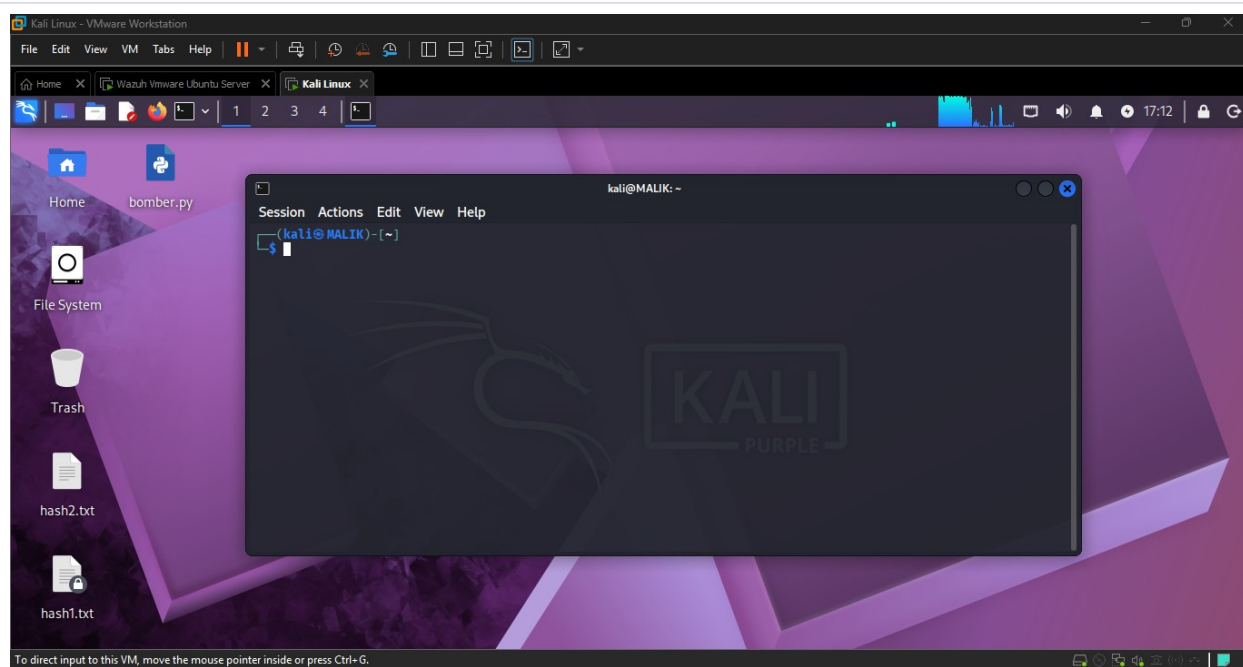


Figure 24. Kali Linux attack host, freshly opened and ready to begin the attack simulation against the Windows endpoint.

Connectivity to the target's SMB service was confirmed first with a targeted port scan.

COMMAND

```
nmap -p 445 <target-ip>
```

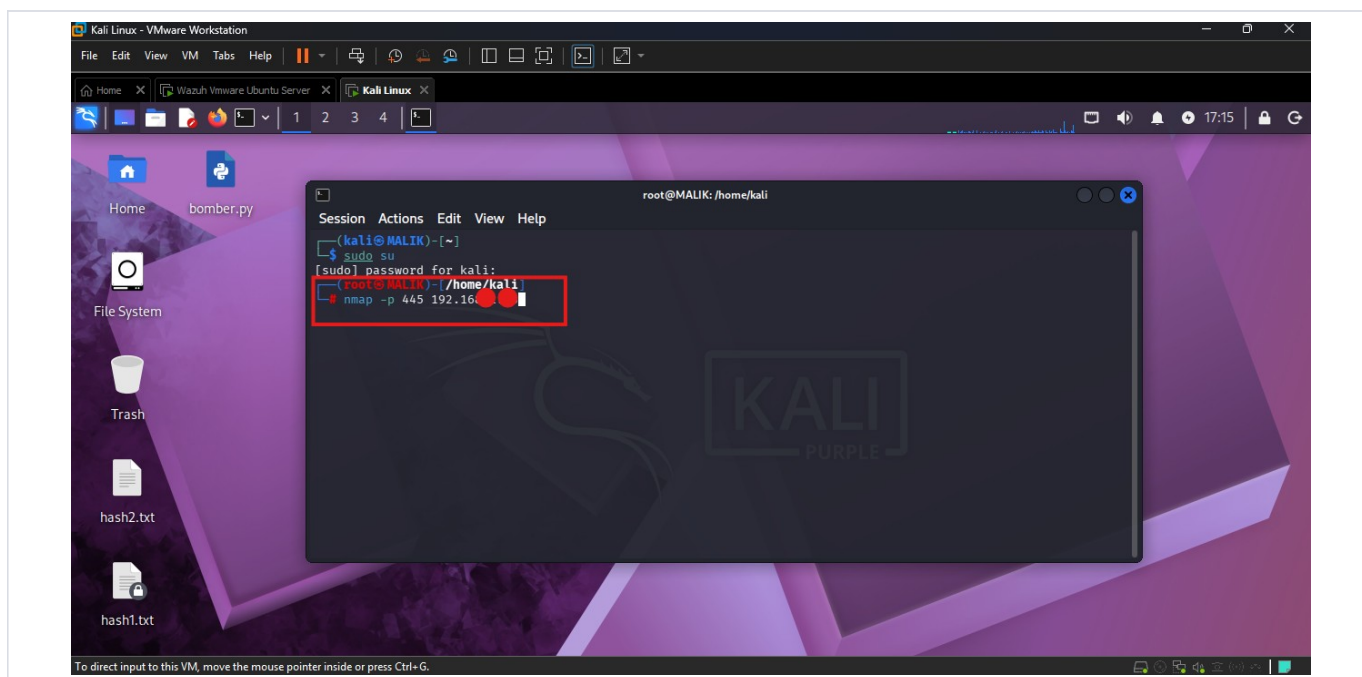


Figure 25. Launching the initial nmap scan against the Windows endpoint's SMB port from Kali.

The scan confirmed port 445 (SMB / microsoft-ds) open and reachable, with fast local-network latency confirming the two hosts were correctly routed to each other.

```
(root@MALIK)-[/home/kali]
# nmap -p 445 192.168.44.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-19 09:20 -0400
Nmap scan report for 192.168.44.1
Host is up (0.00027s latency).

PORT      STATE SERVICE
445/tcp   open  microsoft-ds
MAC Address: 00:50:56:C0:00:08 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.70 seconds

(root@MALIK)-[/home/kali]
#
```

Figure 26. Nmap results confirming port 445/tcp open on the Windows endpoint, ready for the brute-force attempt.

An initial attempt with Hydra failed due to a protocol-compatibility issue with modern SMB2/3, so the attack was switched to NetExec, which handles current Windows SMB versions correctly, targeting the local account with a common password wordlist.

COMMAND

```
netexec smb <target-ip> -u Malik -p /usr/share/wordlists/rockyou.txt
```

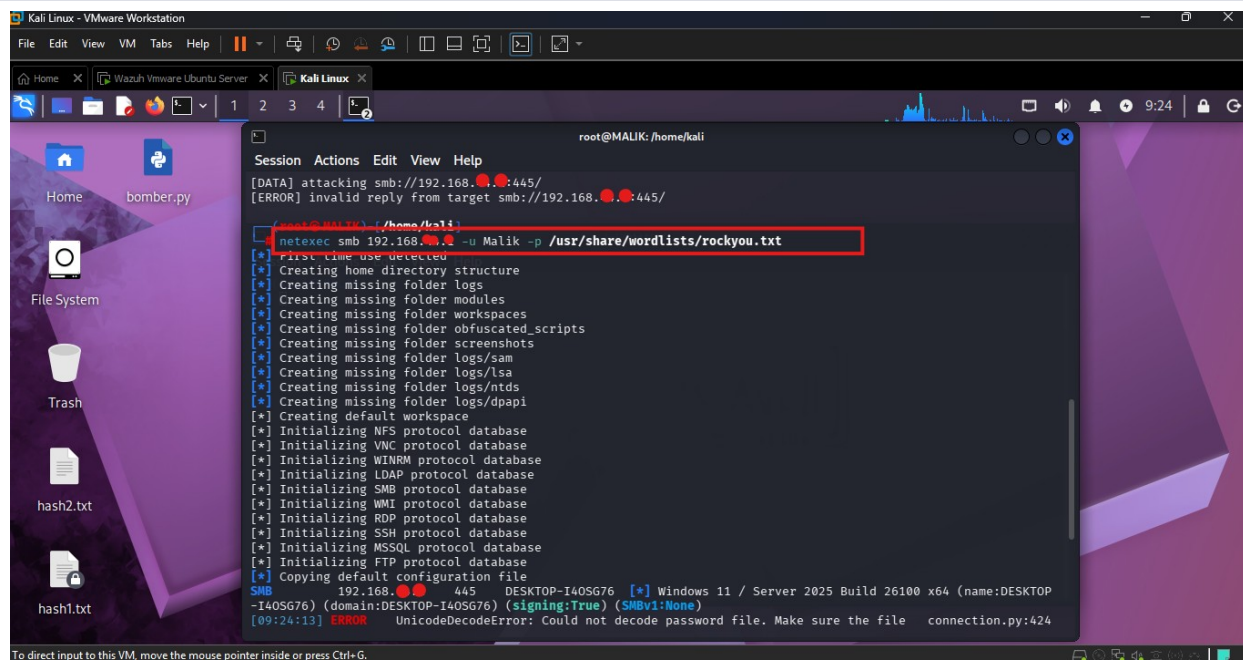


Figure 27. NetExec launched against the Windows endpoint's SMB service after an initial Hydra attempt failed on SMB protocol compatibility.

08 Detection & Automated Response

The brute-force attempt produced an immediate, layered detection chain in the dashboard: suspicious command-shell activity, an abnormal process starting a command prompt, and — most notably — a successful anonymous NTLM logon flagged as a possible pass-the-hash pattern, all within the same second.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 19, 2026 @ 18:40:20.881	Khizar-Malik-Windows11	Suspicious Windows cmd shell execution	3	92032
Jul 19, 2026 @ 18:40:20.879	Khizar-Malik-Windows11	Suspicious Windows cmd shell execution	3	92032
Jul 19, 2026 @ 18:40:20.875	Khizar-Malik-Windows11	Windows command prompt started by an abnormal process	4	92052
Jul 19, 2026 @ 18:40:20.533	Khizar-Malik-Windows11	Host Blocked by route-null Active Response	3	655
Jul 19, 2026 @ 18:40:17.701	Khizar-Malik-Windows11	Logon Failure - Unknown user or bad password	5	60122
Jul 19, 2026 @ 18:40:17.671	Khizar-Malik-Windows11	Windows User Logoff	3	60137
Jul 19, 2026 @ 18:40:17.657	Khizar-Malik-Windows11	Successful Remote Logon Detected - User: \ANONYMOUS LOGON - NTLM authentication, possible pass-the-hash attack.	6	92652

Figure 28. Detection chain from the attack: suspicious cmd shell execution, an abnormal process spawning a command prompt, and a successful anonymous NTLM logon flagged as a possible pass-the-hash attack.

Expanding the Active Response event itself confirmed the automated block firing (data.command: "add"), correlated with the underlying authentication data: the attack originated from the Kali host at 192.168.44.128 over NTLM, matching exactly the rule pair the response was bound to.

Field	Value
@timestamp	Jul 19, 2026 @ 18:48:28.533
_index	wazuh-alerts-4.x-2026.07.19
agent.id	001
agent.ip	192.168.44.1
agent.name	Khizar-Malik-Windows11
data.command	add
data.origin.module	wazuh-execd
data.origin.name	node01
data.parameters.alert.agent.id	001
data.parameters.alert.agent.ip	192.168.44.1
data.parameters.alert.agent.name	Khizar-Malik-Windows11
data.parameters.alert.data.win.eventdata.authenticationPackageName	NTLM
data.parameters.alert.data.win.eventdata.failureReason	%2313
data.parameters.alert.data.win.eventdata.ipAddress	192.168.44.128
data.parameters.alert.data.win.eventdata.ipPort	58298
data.parameters.alert.data.win.eventdata.keyLength	0
data.parameters.alert.data.win.eventdata.logonProcessName	NtLmSsp
data.parameters.alert.data.win.eventdata.logonType	3
data.parameters.alert.data.win.eventdata.processId	0x0
data.parameters.alert.data.win.eventdata.status	0xc000006d

Figure 29. Active Response event detail showing the block command ("add") fired by wazuh-execd, correlated with the NTLM authentication data from the Kali source IP.

The underlying raw Windows Security event was also reviewed directly, confirming the full authentication failure detail: the targeted account, the failure reason, and the exact source IP and port the attack came from.

Field	Value
data.parameters.alert.data.win.system.message	*An account failed to log on.
Subject:	Security ID: S-1-0-0 Account Name: - Account Domain: - Logon ID: 0x0
Logon Type:	3
Account For Which Logon Failed:	Security ID: S-1-0-0 Account Name: Malik Account Domain: DESKTOP-1405076
Failure Information:	Failure Reason: Unknown user name or bad password Status: 0xc000006d Sub Status: 0xc000006a
Process Information:	Caller Process ID: 0x0 Caller Process Name: -
Network Information:	Workstation Name: - Source Network Address: 192.168.44.128 Source Port: 58298
Detailed Authentication Information:	Logon Process: NtLmSsp Authentication Package: NTLM Transmitted Services: - Package Name (NTLM only): - Key Length: 0

This event is generated when a logon request fails. It is generated on the computer where access was attempted.

Figure 30. The raw Windows Security event ("An account failed to log on") showing the targeted account and the attacking source IP and port.

09 Full Detect-to-Block Lifecycle

A final review of the dashboard's event timeline confirmed the complete, correctly-ordered lifecycle: the attack's command-shell and logon-failure activity, the Active Response block firing, and — after the configured timeout — the automatic unblock, all timestamped and traceable in sequence.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 19, 2026 @ 18:50:20.851	Khizar-Malik-Windows11	Host Unblocked by route-null Active Response	3	656
Jul 19, 2026 @ 18:49:57.749	Khizar-Malik-Windows11	Executable file dropped in folder commonly used by malware	15	92213
Jul 19, 2026 @ 18:49:52.743	Khizar-Malik-Windows11	Sysmon - Suspicious Process - dlhhost.exe	12	61638
Jul 19, 2026 @ 18:49:52.721	Khizar-Malik-Windows11	Sysmon - Suspicious Process - dlhhost.exe	12	61638
Jul 19, 2026 @ 18:49:37.508	Khizar-Malik-Windows11	Executable file dropped in folder commonly used by malware	15	92213
Jul 19, 2026 @ 18:40:20.881	Khizar-Malik-Windows11	Suspicious Windows cmd shell execution	3	92032
Jul 19, 2026 @ 18:40:20.879	Khizar-Malik-Windows11	Suspicious Windows cmd shell execution	3	92032
Jul 19, 2026 @ 18:40:20.875	Khizar-Malik-Windows11	Windows command prompt started by an abnormal process	4	92052
Jul 19, 2026 @ 18:40:20.533	Khizar-Malik-Windows11	Host Blocked by route-null Active Response	3	655
Jul 19, 2026 @ 18:40:17.701	Khizar-Malik-Windows11	Login Failure - Unknown user or bad password	5	60122
Jul 19, 2026 @ 18:40:17.671	Khizar-Malik-Windows11	Windows User Logoff	3	60137
Jul 19, 2026 @ 18:40:17.657	Khizar-Malik-Windows11	Successful Remote Logon Detected - User:ANONYMOUS LOGON - NTLM authentication, possible pass-the-hash attack.	6	92652

Figure 31. Full event timeline confirming the complete detect-to-block lifecycle for the simulated attack, from initial activity through the automated Active Response and its eventual timeout release.

Result

The full EDR loop was demonstrated end-to-end: a live brute-force attack was detected within milliseconds, automatically blocked via Wazuh's Active Response, and released again after its configured timeout — without any manual intervention.

10 MITRE ATT&CK Mapping

The techniques simulated and detected throughout this lab, mapped against the MITRE ATT&CK framework:

Technique ID	Technique Name	Tactic	Evidence in this Lab
T1547.001	Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder	Persistence	Renamed executable dropped into the Windows Startup folder; caught by real-time File Integrity Monitoring (Rules 553 / 554).
T1036	Masquerading	Defense Evasion	calc.exe copied and renamed to totally_legit_update.exe to imitate a legitimate update file.
T1110	Brute Force	Credential Access	SMB authentication brute-forced from Kali against the Windows endpoint using NetExec, generating repeated logon-failure alerts (Rule 60122).
T1557 / T1075	Adversary-in-the-Middle / Pass-the-Hash (NTLM)	Lateral Movement / Credential Access	Anonymous NTLM logon detected and flagged as a possible pass-the-hash pattern (Rule 92652) during the brute-force attempt.

11 Key Findings & Lessons Learned

Building working detection and response coverage surfaced two practical engineering lessons worth documenting, since resolving them accurately — rather than around them — is the core skill this lab exercises.

#	Issue Diagnosed	Root Cause	Resolution
1	A file deliberately copied into the Startup folder during the persistence simulation did not appear as a Sysmon-driven alert.	PowerShell's Copy-Item cmdlet writes files through a different underlying Windows API path than a typical dropper uses, which process-level Sysmon rules are not guaranteed to match.	Relied on File Integrity Monitoring (FIM/syscheck) instead — FIM is hash- and timestamp-based and is agnostic to how a file was written, so it caught the change regardless of the tooling used to create it.
2	The Active Response was initially at risk of never firing against the Windows endpoint.	Wazuh's generic Linux/PAM brute-force rule ID (5712) does not apply to Windows agents — Windows authentication failures are scored under separate, platform-specific rule IDs.	Bound the win_route-null Active Response directly to Wazuh's Windows-specific logon-failure rules (60122 "Logon Failure" and 60115 "Account Locked Out") instead of the generic Linux rule ID.

Additional observations

- A copy operation performed with PowerShell's Copy-Item does not always trigger the same process-level detections as a genuine dropper would — File Integrity Monitoring is a necessary, sensor-agnostic complement that catches the resulting file-system change regardless of how it was written.
- Wazuh's Windows-specific authentication-failure rules (60122, 60115) are distinct from the generic Linux/PAM brute-force rule (5712) — active-response bindings must target the platform-correct rule IDs to actually fire.
- Correlating an Active Response event's own record (data.command: "add") against the underlying raw Windows Security event gives a complete, defensible evidence trail from detection through to automated action.

12 Conclusion

This lab took a Wazuh deployment from passively receiving endpoint telemetry to actively detecting and automatically responding to a live, simulated attack. Sysmon health and forwarding were verified first, File Integrity Monitoring was configured and proven against a realistic persistence attempt, and an Active Response was engineered, correctly bound to platform-specific detection rules, and validated against a genuine SMB brute-force attack launched from a separate attack host — with the complete detect-to-block-to-release lifecycle captured as evidence. This workflow — verify sensor health, configure detection, engineer a response, and validate it against a real test — mirrors the process a SOC analyst uses when building and tuning a live detection stack.